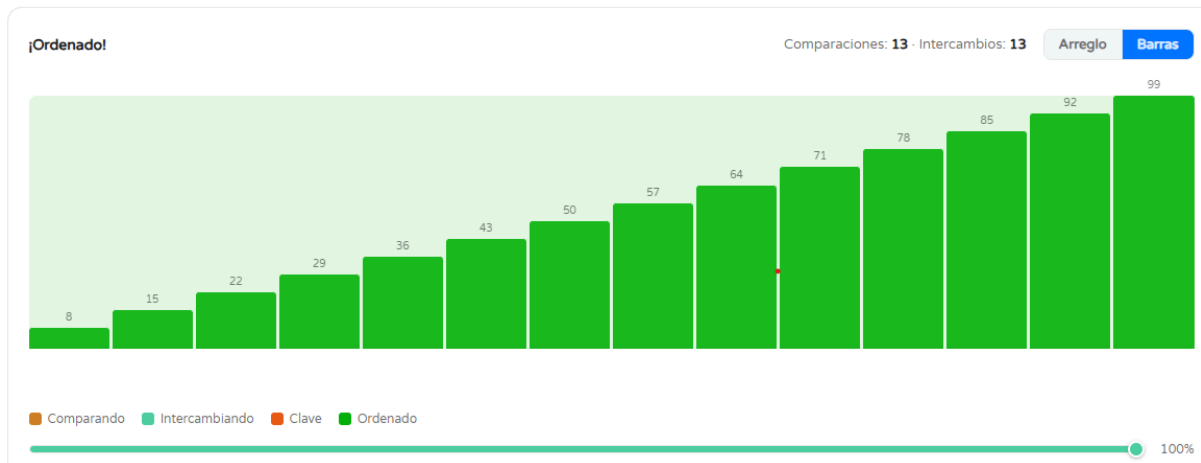


ALGORITMOS DE ORDENACIÓN: INSERTION SORT

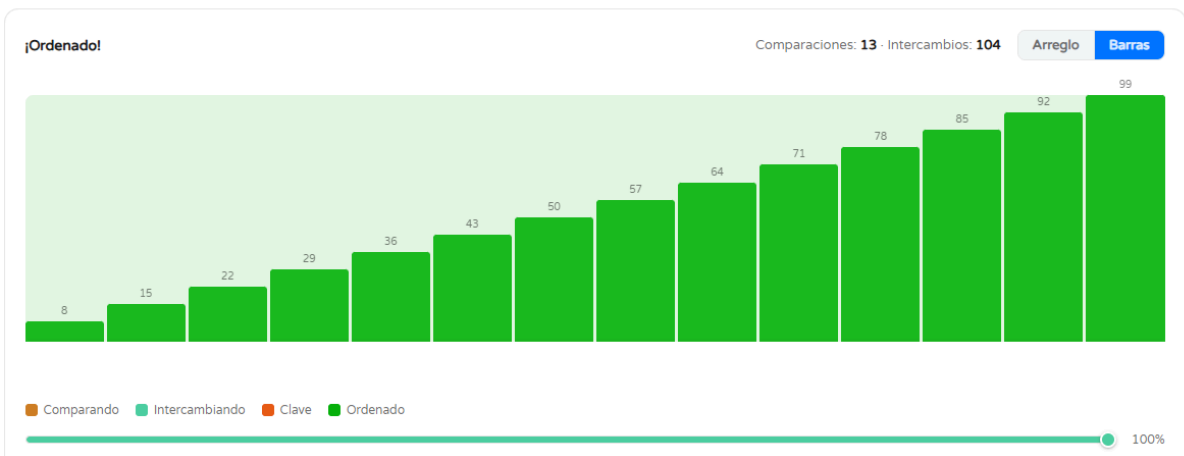
Complejidad en el mejor caso.

$O(n)$ Ya ordenado.



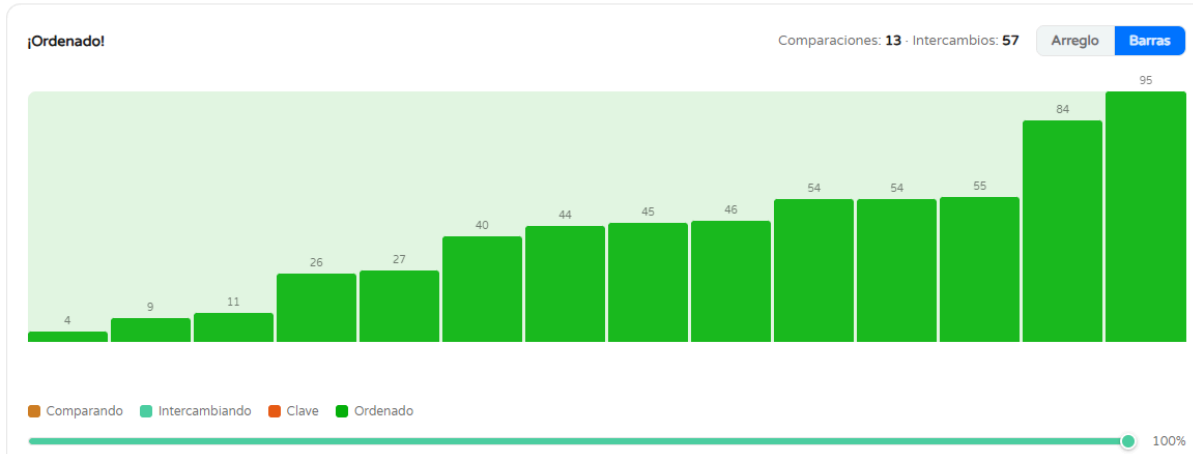
Complejidad en el peor caso.

$O(n^2)$ Orden inverso.



Complejidad en el caso promedio.

$O(n^2)$ Orden aleatorio.



¿Es adecuado para Big Data? ¿Por qué?

No, ya que el ordenamiento por inserción es muy rápido con entradas pequeñas o casi ordenadas y no con grandes volúmenes de datos.

Cuándo usar el ordenamiento por inserción.

Úsalo cuando	Evítalo cuando
El arreglo es pequeño (aproximadamente $n < 20$).	El arreglo es grande y está ordenado al azar.
Los datos ya están casi ordenados, dando el mejor caso $O(n)$.	Necesitas un peor caso garantizado de $O(n \log n)$.
Necesitas un ordenamiento estable, in situ y con espacio extra $O(1)$.	Mover elementos es costoso, ya que realiza muchos desplazamientos.
Los datos llegan de forma incremental y deben mantenerse ordenados en línea.	La entrada está en orden inverso, su peor caso $O(n^2)$.

Explicado con un ejemplo de la vida cotidiana sería como ordenar una baraja de cartas en donde se va insertando en el orden indicado.

Fuentes: <https://coddy.tech/visualize/es/sorting/insertion-sort>